

Motor de Recomendación Turística con IA — Visión del Proyecto

1. Contexto y Necesidad (TUI)

El sector turístico enfrenta una paradoja: los sistemas de recomendación actuales, diseñados para maximizar conversión, terminan concentrando la demanda en los mismos destinos populares de siempre. El resultado es bien conocido: overtourism en Barcelona, Santorini o Cancún, saturación de infraestructuras, tensión con las comunidades locales y una distribución muy desigual del gasto turístico.

TUI, como operador turístico global, tiene un doble interés en resolver este problema. Por un lado, es una responsabilidad de sostenibilidad. Por otro, es una oportunidad de negocio: los destinos menos saturados tienen capacidad disponible, precios más competitivos y, en muchos casos, una experiencia de viaje superior para el cliente.

El objetivo concreto de TUI es redistribuir los flujos turísticos hacia destinos menos saturados, temporadas bajas y productos infrautilizados, sin que eso implique un sacrificio en la satisfacción del viajero. El público objetivo es el viajero premium, con preferencia por paquetes all-inclusive (vuelo + hotel + destino), especialmente en el Mediterráneo y el Caribe.

2. Objetivo del Proyecto

Se va a construir un motor de recomendación con inteligencia artificial que combine tres dimensiones: personalización del viajero, criterios de sostenibilidad y redistribución inteligente de la demanda turística. El sistema incluirá una aplicación para el usuario final, donde el viajero recibirá recomendaciones personalizadas explicadas en lenguaje natural, y un dashboard analítico interno para TUI, con métricas de redistribución, saturación por destino y oportunidades de mercado.

El resultado del proyecto es un prototipo funcional, diseñado y documentado como Trabajo de Fin de Máster, con rigor académico y orientación práctica.

3. Estructura del Proceso (Pipeline)

El sistema se articula en seis bloques que van desde la obtención de datos hasta las interfaces de usuario.

Bloque 1 — Scraping y Limpieza de Datos

Qué se hace: Extraer datos de paquetes turísticos de TUI (destinos, precios, disponibilidad, categorías, ocupación, temporadas) y complementarlos con fuentes públicas que aporten indicadores de saturación, reseñas de viajeros y tendencias de demanda.

De dónde: La web de TUI, con foco en el catálogo de paquetes del Mediterráneo y el Caribe. Como fuentes públicas: TripAdvisor, foros de viajeros, datos abiertos de organismos de turismo (Eurostat, OMT, INE).

Cómo: Web scraping con Python (BeautifulSoup, Scrapy o Selenium según la complejidad de cada fuente). Los datos se refresca de forma periódica, con ciclos configurables.

Limpieza: Deduplicación de registros, normalización de campos numéricos (escalado min-max), estandarización de categorías y exclusión de registros incompletos.

Preguntas abiertas:

- ¿Qué fuentes públicas concretas usar para datos de saturación turística?
- ¿Con qué frecuencia refrescar los datos (diario, semanal)?

Bloque 2 — Embeddings y NLP

Qué se hace: Convertir los textos extraídos (descripciones de paquetes, reseñas de usuarios) en vectores semánticos que permitan comparar similitudes de significado más allá de palabras exactas.

Cómo: Modelos de embeddings preentrenados. Los candidatos principales son Sentence-BERT, paraphrase-multilingual y text-embedding-ada-002. La representación final de cada paquete será híbrida: vector semántico concatenado con atributos estructurados normalizados (precio, duración, categoría, etc.).

Preguntas abiertas:

- ¿Qué modelo de embeddings ofrece el mejor equilibrio entre calidad y coste computacional?
- ¿Cómo gestionar textos en múltiples idiomas (español, inglés, alemán)?

Bloque 3 — Estructura de Datos y Data Engineering

Qué se hace: Centralizar toda la información en una base de datos con tres entidades principales: Usuarios, Experiencias (Paquetes) e Interacciones. Esta estructura es la base sobre la que opera el modelo recomendador.

Dónde: Las opciones en evaluación son Azure SQL / Cosmos DB (candidato natural dado el contexto corporativo de TUI), PostgreSQL o SQLite para el prototipo de TFM. Para búsqueda vectorial, se evalúan pgvector, Qdrant y Chroma.

Cómo se sirven los datos: Una API REST construida con FastAPI actuará como capa de acceso desde el modelo y desde las interfaces. El procesamiento de embeddings y scores masivos se hará en batch.

Preguntas abiertas:

- ¿Azure o solución local para el entorno del TFM?
- ¿Qué motor usar para búsqueda vectorial eficiente?

Bloque 4 — Modelo Recomendador

Qué se hace: Calcular la afinidad entre cada usuario y cada paquete disponible, y generar un ranking personalizado que incorpore criterios de redistribución turística.

Opciones a evaluar:

- Baseline: similitud del coseno, KNN
- Modelos avanzados: LightFM (factorización matricial), XGBoost (features tabulares), Two-Tower (redes neuronales duales)

Componentes propios del sistema:

- **Modelo de Afinidad:** Puntuación de compatibilidad entre usuario y experiencia, en el rango [0, 1].
- **TDRS (Tourism Demand Redistribution Score):** Índice compuesto que combina afinidad del usuario, capacidad disponible del destino, accesibilidad, impacto local, temporada, ocupación actual y sensibilidad ambiental.
- **Re-ranking dinámico:** La puntuación final de cada recomendación se calcula como:

$\text{Score_Final} = \alpha \cdot \text{Base} + \beta \cdot \text{Redistribución} + \gamma \cdot \text{Sostenibilidad} + \delta \cdot \text{Capacidad} - \lambda \cdot \text{Saturación}$

- **Tres escenarios de recomendación:** Tradicional (máxima afinidad), redistribución moderada y redistribución intensiva. El usuario puede comparar los tres.

Preguntas abiertas:

- ¿Cuál modelo avanzado implementar y comparar contra el baseline?
- ¿Cómo generar suficientes datos de interacción para entrenamiento cuando no hay historial real disponible (usuarios sintéticos)?

Bloque 5 — Integración con LLM

Qué se hace: Usar un modelo de lenguaje grande (LLM) para transformar el ranking de recomendaciones en texto natural personalizado para el usuario, haciendo las recomendaciones comprensibles y atractivas.

Enfoque: No se construye un LLM desde cero. Se usa un modelo público (candidatos: GPT-4o-mini, Llama 3, Mistral) adaptado mediante prompt engineering con el contexto del usuario y del paquete recomendado.

Funcionalidades concretas:

- Generar descripciones personalizadas de los tres paquetes mejor puntuados
- Incluir referencias explícitas a las preferencias del usuario (gastronomía, naturaleza, presupuesto, tipo de viaje)
- Mencionar el beneficio de sostenibilidad cuando el TDRS del paquete es alto
- Fallback a plantillas predefinidas si el LLM no está disponible

Detección de oportunidades: El sistema identificará destinos infrautilizados con potencial de demanda, generando un listado útil para campañas comerciales de TUI.

Preguntas abiertas:

- ¿Qué LLM usar y bajo qué criterios (coste, calidad, privacidad)?
- ¿Despliegue local con Ollama o vía API en la nube?

- ¿Cómo controlar el coste de las llamadas a la API del LLM?

Bloque 6 — Productivización

Qué se construye: Dos interfaces diferenciadas.

1. **App de usuario final:** El viajero introduce su perfil (preferencias, presupuesto, fechas, tipo de viaje) y recibe recomendaciones personalizadas. Las recomendaciones incluyen una explicación en lenguaje natural generada por el LLM. El usuario puede cambiar entre los tres escenarios de recomendación y comparar resultados.

2. **Dashboard analítico para TUI:** Vista interna con métricas de redistribución turística, diversidad del catálogo recomendado, saturación por destino, comparativa entre escenarios y listado de oportunidades de mercado detectadas. La actualización es periódica, no en streaming.

Stack candidato: Streamlit para el prototipo del TFM (desarrollo rápido, suficiente para la demo académica). FastAPI + React como alternativa si se quiere una arquitectura más desacoplada y mantenible.

Preguntas abiertas:

- ¿Streamlit es suficiente o conviene separar backend y frontend?
- ¿Se explora un bot de WhatsApp como canal adicional de recomendación?

4. Decisiones Tomadas

Las siguientes decisiones están confirmadas y no requieren revisión adicional:

- **Lenguaje de programación:** Python en todo el stack
- **Origen de datos:** Híbrido — scraping real de TUI + fuentes públicas + usuarios sintéticos para entrenamiento
- **Alcance de la entrega:** Prototipo funcional, no un sistema productivo a escala industrial
- **Interfaces:** App de usuario final + Dashboard analítico para TUI
- **Refresco de datos:** Periódico (no tiempo real estricto)
- **Modelo recomendador:** Al menos un baseline y un modelo avanzado, con comparativa documentada
- **Escenarios de recomendación:** Tres escenarios (tradicional, redistribución moderada, redistribución intensiva)

5. Decisiones Pendientes

Estas decisiones deben tomarse antes de entrar en el diseño técnico detallado:

- Motor de base de datos (relacional vs vectorial, Azure vs solución local)
- Modelo de embeddings específico a usar
- Modelo LLM concreto (y modalidad de despliegue: local u API)
- Modelo avanzado de recomendación a implementar

- Stack de productivización (Streamlit vs FastAPI + frontend separado)
- Fuentes de datos públicas concretas para indicadores de saturación y reseñas de viajeros

6. Métricas de Éxito

El sistema se evaluará con las siguientes métricas, agrupadas por dimensión:

Calidad de la recomendación:

- Precision@K — proporción de ítems relevantes entre los K recomendados
- Recall@K — cobertura de los ítems relevantes en el top K
- NDCG@K — calidad del ranking teniendo en cuenta la posición
- MAP@K — precisión media sobre todos los usuarios

Diversidad del catálogo:

- Intra-list diversity — variedad dentro de cada lista de recomendaciones
- Cobertura del catálogo — qué porcentaje del catálogo total aparece recomendado
- Novedad — en qué medida se recomiendan destinos poco conocidos para el usuario

Redistribución turística:

- Coeficiente Gini turístico — desigualdad en la distribución de demanda entre destinos
- CR5 — concentración en los cinco destinos más recomendados
- Reducción de saturación media — comparativa entre escenarios
- Porcentaje de demanda dirigida a destinos menos visitados — métrica clave de impacto

Documento de visión y alcance — Motor de Recomendación Turística con IA — TFM